INTERNSHIP PROJECT REPORT

Advanced Port Scanner Using Python

Submitted by: Samyak Bhaisare

Year: 2026

1. Introduction

Port scanning is a fundamental technique in cybersecurity used to identify open ports and services running on a target system. This project implements an advanced port scanner using Python with multithreading and risk classification.

2. Objectives

- Identify open ports
- Detect services
- Implement multithreading
- Add risk tagging
- Provide scan modes
- Save results

3. Tools and Technologies Used

- Python
- Socket Programming
- ThreadPoolExecutor
- Kali Linux

4. Methodology

1. User inputs target
2. Selects scan mode
3. Ports scanned using threading
4. Open ports detected
5. Risk tagging applied
6. Results saved

5. Results

Example Output:

Port 22 → SSH → Risk: Medium

The scanner successfully detected open ports using both Quick Scan and Custom Scan modes, along with their associated services and risk levels.

```
┌──(kali㉿Kali)-[~/Desktop/port_scaneer]
└─$ python port_scaneer.py
═══════════════════════════════════════════════════════════
      ADVANCED PORT SCANNER
   Threading + Service Detection + Risk Tagging
═══════════════════════════════════════════════════════════
Enter Target IP or Domain: 127.0.0.1

Select Scan Mode:
1. Quick Scan (Common Ports)
2. Custom Scan Range
Enter your choice (1/2): 1

Scanning Target: 127.0.0.1 (127.0.0.1)
Scan Type: Quick Scan
───────────────────────────────────────────────────────────
[OPEN] Port 22 | Service: ssh | Risk: MEDIUM
───────────────────────────────────────────────────────────
Total Ports Scanned: 14
Open Ports Found: 1
Scan Completed At: 2026-04-26 12:24:02.128045
```

Caption: Figure 1 – Quick Scan Output

```
┌──(kali㉿Kali)-[~/Desktop/port_scaneer]
└─$ python port_scaneer.py
═══════════════════════════════════════════════════════════
      ADVANCED PORT SCANNER
   Threading + Service Detection + Risk Tagging
═══════════════════════════════════════════════════════════
Enter Target IP or Domain: 127.0.0.1

Select Scan Mode:
1. Quick Scan (Common Ports)
2. Custom Scan Range
Enter your choice (1/2): 2
Enter Start Port: 1
Enter End Port: 1000

Scanning Target: 127.0.0.1 (127.0.0.1)
Scan Type: Custom Scan
───────────────────────────────────────────────────────────
[OPEN] Port 22 | Service: ssh | Risk: MEDIUM
───────────────────────────────────────────────────────────
Total Ports Scanned: 1000
Open Ports Found: 1
Scan Completed At: 2026-04-26 12:24:37.381346
```

Caption: Figure 2 – Custom Scan Output

```
┌──(kali㉿Kali)-[~/Desktop/port_scaneer]
└─$ cat scan_results.txt
ADVANCED PORT SCANNER RESULTS
═══════════════════════════════════════════════════════════════
Target: 127.0.0.1
Target IP: 127.0.0.1
Scan Type: Custom Scan
Scan Time: 2026-04-26 12:24:37.271905 to 2026-04-26 12:24:37.381346
───────────────────────────────────────────────────────────────
[OPEN] Port 22 | Service: ssh | Risk: MEDIUM
───────────────────────────────────────────────────────────────
Total Ports Scanned: 1000
Open Ports Found: 1
```

Caption: Figure 3 – Saved Results File

6. Applications

- Network analysis
- Security testing
- Learning

7. Ethical Considerations

Use only on authorized systems.

8. Conclusion

The project successfully demonstrates efficient port scanning using multithreading, along with risk classification and flexible scan modes. This makes the tool practical and useful for basic cybersecurity analysis.